

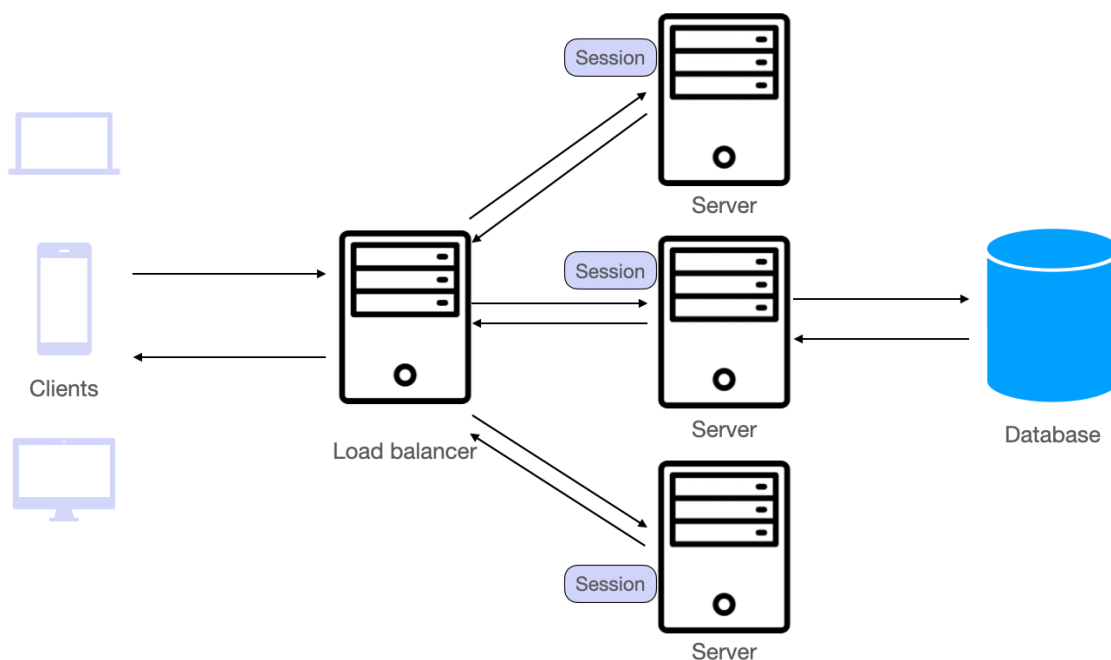
Stateless vs Stateful

🌐 systemdesignschool.io/fundamentals/evolution-of-a-web-app-stateless-vs-stateful

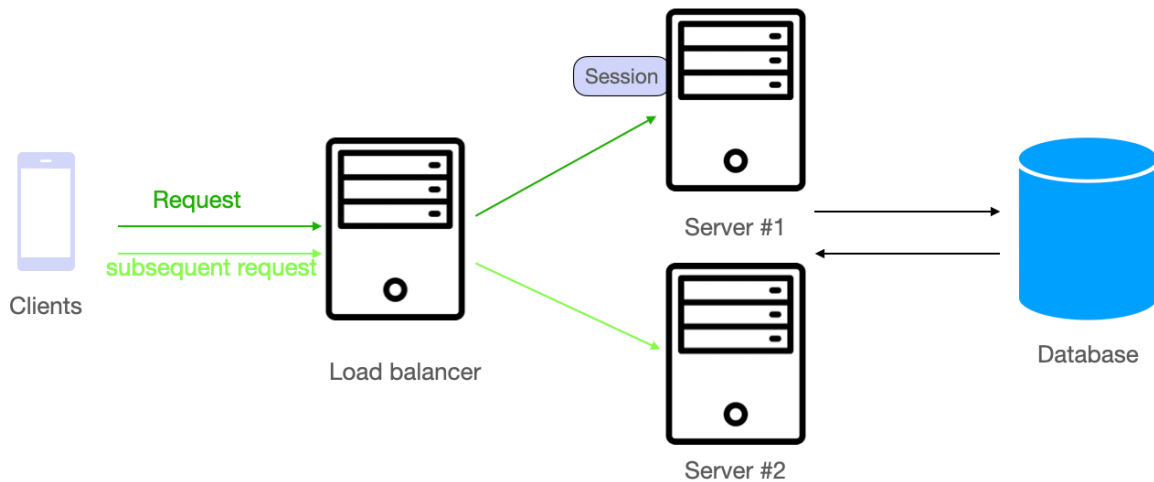


Evolution of a Web App | Stateless vs Stateful

While everything appears to be progressing smoothly, we must confront the challenge of managing state. Given that this is your uncle's e-commerce store, dealing with shopping carts is a must. These carts are commonly implemented using user sessions, which are in turn built on in-memory cookies. This is where we encounter a potential problem.



Consider the following scenario: a user visits the site and adds a bag of dog food to their cart. Server #1 handles this request, generating a new cookie for the cart. However, the subsequent request might be processed by Server #2, which has no knowledge of the initial cookie. The result is a frustrated user whose items have inexplicably disappeared from their shopping cart, creating a less-than-ideal experience.



Stateful

The issue is that we are storing session data (aka "state") in each server, and therefore the servers are "stateful." How do we go about solving that?

One option is to configure the load balancer to use "session affinity" or "sticky sessions." This involves configuring the load balancer to forward requests from a particular user to a specific server for the duration of their session. By doing so, the server maintains the session state and can continue to process requests for that session.

This works, but there are some downsides. For example, if the server fails, the session data stored on that server may be lost. Additionally, this approach can make it challenging to scale horizontally because it requires the load balancer to maintain state information about each user's session and map it to a specific server. Finally, it can make it difficult to distribute traffic evenly across servers, which may result in some servers becoming overloaded while others are underutilized.

Stateless

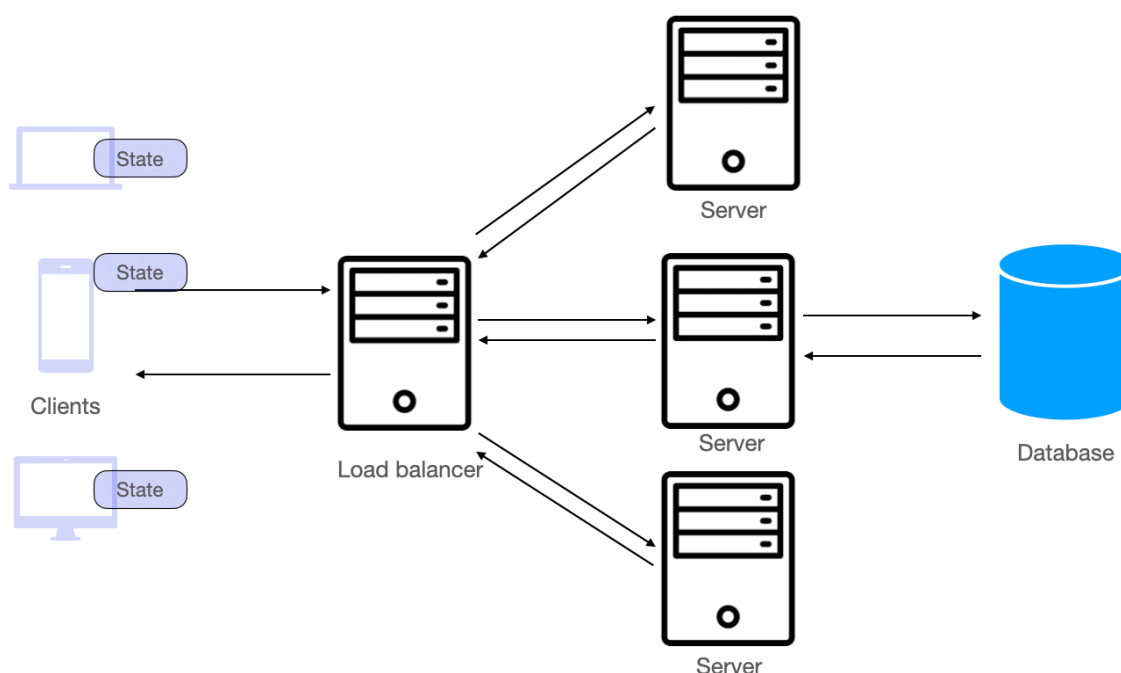
Now, what about removing state from the web servers and making them "stateless"? There are a few ways to go about this.

Keeping States On the Client

We could move the shopping cart state to the client (e.g., the user's browser or mobile app). One way to implement this is by using client-side storage (e.g., Local Storage/IndexedDB for browsers, Core Data for iOS devices etc). Here's how it works:

1. When a user adds an item to their cart, the browser stores the cart data in the client-side storage.
2. With each subsequent request, the browser sends the shopping cart data to the server, either as part of the request or in a custom header.
3. The server processes the request based on the provided cart data (e.g., calculating the total or applying discounts) and sends the response back to the client.

In this stateless approach, the server does not maintain any knowledge of the shopping cart between requests, relying on the client to manage the state.



How to Store Session On the Client:

1. Cookies:
 - Use Cases: Ideal for session management, user authentication, and storing user preferences. For example, an e-commerce site might use cookies to remember items in a shopping cart or a user's language settings.
 - Pros: Supported by all browsers and automatically included in HTTP requests.
 - Cons: Limited in size (about 4KB), and can pose security risks if not properly handled (vulnerable to CSRF attacks).
 - When to Use: Best for small pieces of data where persistence across sessions is needed and when server-side access to this data is required.

2. Local Storage and Session Storage:

- Use Cases: Local storage is great for storing user settings, such as theme preferences or layout settings in a web application. Session storage is useful for temporary data like form inputs in a multi-step form process.
- Pros: Easy to use, with a larger capacity (up to 5-10MB).
- Cons: Local storage data doesn't expire, which can lead to storage management issues. Session storage is limited to the session's duration.
- When to Use: Use local storage for long-term data storage and session storage for data relevant to a single browser session.

3. IndexedDB:

- Use Cases: Ideal for applications needing to store large amounts of data client-side, like offline-first applications. For instance, a note-taking app might use IndexedDB to store notes locally for offline access.
- Pros: Can store large amounts of structured data, works asynchronously.
- Cons: More complex API compared to local or session storage.
- When to Use: Best for complex applications where large amounts of data need to be stored and manipulated client-side.

4. Client-Side JavaScript Libraries:

- Use Cases: Redux is often used in complex React applications for managing global state, like user authentication status or shared user data across components. Vuex serves a similar purpose in Vue.js applications.
- Pros: Centralizes state management, making it easier to manage and debug.
- Cons: Can add complexity to the project, and might be overkill for simple applications.
- When to Use: Ideal for single-page applications (SPAs) with complex state management needs.

5. JWT (JSON Web Tokens):

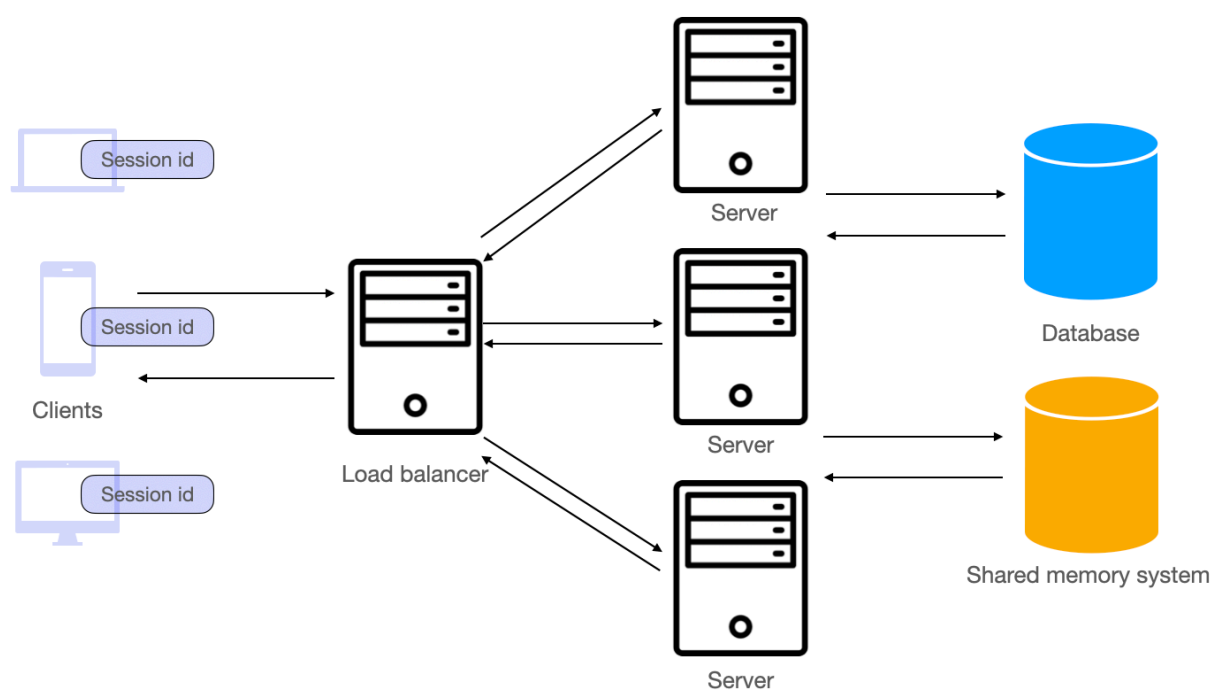
- Use Cases: Commonly used in web applications for securely handling user authentication and transmitting user information. For example, after logging in, a user receives a JWT to make authenticated API requests.
- Pros: Enables stateless authentication, is self-contained, and can be easily transmitted across different domains.
- Cons: Storage can be challenging, and it is vulnerable to XSS attacks if not stored securely.
- When to Use: Best for applications where security, scalability, and cross-domain access are important, especially in RESTful APIs and SPAs.

Server-side Shared Memory

Another way is to have a shared memory system like Memcached or Redis in front of the web servers. Here's how it works:

1. When a user adds an item to their cart, the server creates a session and stores the cart data in a shared memory store such as Memcached or Redis.
2. The server sends a session identifier (e.g., a session cookie) to the user's browser.
3. With each subsequent request, the user's browser sends the session identifier back to the server.
4. The server uses the session identifier to fetch the corresponding session data (i.e., the user's shopping cart) from the shared memory store and performs the requested operations (e.g., adding or removing items).

By using a shared memory store like Memcached or Redis, the session data can be accessed by multiple servers, allowing for better scalability in distributed environments. Additionally, since Memcached and Redis are in-memory data stores, they can offer faster access times compared to traditional databases, improving overall performance.



How to Store State in Shared Memory

Redis and Memcached are typically the go-to technologies for in-memory session storage. Cloud providers often offer fully managed distributed caching services. For example, AWS offers ElasticCache for Redis and Memcached.

Which One to Use in Scalable Web Applications

Stateless web server setup is what most modern production systems use. Compute is typically the cheaper resource, and we can scale it virtually limitlessly. We can even set up systems that auto-scales the stateless servers so that when demands are high we launch more servers.